

Calyr_ai_syntax: Concept and Architecture

A symbolic language and execution layer for SPR, SAXS, Nexus, and
predictive scientific workflows
small BOKU Launch / May 2026

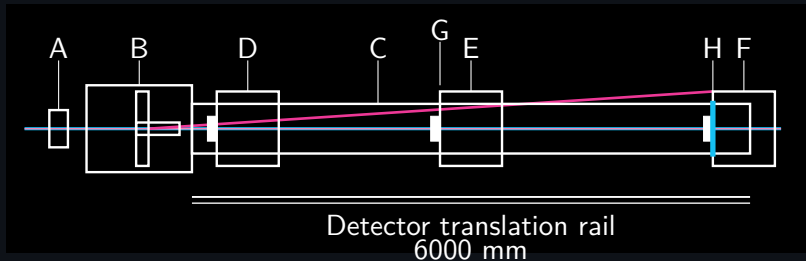
CALYRAI × BOKU VIENNA

Institute of Biophysics / University of Natural Resources and
Life Sciences, Vienna

1 May 2026

symbolic language / Nexus control plane / SPR + SAXS + PCA integration

Instrument Layout



A: Collimators. B: Sample chamber with two sample positions. C: 6 m vacuum flight tube.

D: WAXS detector (beamstop). E: SAXS detector (beamstop). F: USAXS detector (beamstop).

G: PSL (Primary Scattering Line). H: Cyan Q-vector (scattering direction marker).

Cyan = direct X-ray beam. Magenta = scattering trajectory. Detectors translate along the 6000 mm rail to set the accessible q -range.

Problem

- ▶ Scientific code tends to fracture across notebooks, scripts, and engine-specific wrappers.
- ▶ The semantic object is usually lost once execution starts.
- ▶ Calyr needs a syntax layer that is symbolic first and executable second.

Design stance

Reduced surface.

Explicit structure.

Engine-agnostic semantics.

Core Idea

Definition

Calyr_ai_syntax is a Mathematica-inspired symbolic expression layer for Python-centric computational workflows.

EXPRESS

Trees, rules, symbolic forms, explicit operators.

MAP

Bind abstract forms to engines, datasets, and pipelines.

RUN

Preserve structure through execution, logging, and output manifests.

Two Sample Regimes

Downloaded cohort

A broad sample set built from public protein SAXS entries, each carrying measured $I(q)$, optional $P(r)$, metadata, and model attachments.

- ▶ many proteins
- ▶ one measurement-centric structure
- ▶ ideal for representation learning and PCA

Structured sample

A deeper sample object such as IgG, where AlphaFold, cryo-EM, LAMMPS, and SAXS belong to one coordinated system state.

- ▶ one system, many modalities
- ▶ one object identity across stages
- ▶ ideal for integrative evaluation

Downloaded Protein Cohort

Cohort example

Public SASBDB protein entries such as SASDXF6, SASDYY5, and SASDC44 provide experimental $I(q)$, optional $P(r)$, metadata, and model files in one reproducible layout.

- ▶ Each entry is a valid scientific sample object.
- ▶ The first stable representation is the measured scattering curve $I(q)$.
- ▶ Optional model and $P(r)$ files remain attached as context, not mandatory preprocessing.

Sample Structure: IgG

Reference sample object

IgG is the structured reference case: one protein system tied simultaneously to AlphaFold structure, cryo-EM constraints, LAMMPS simulation, and SAXS observables.

- ▶ The sample is not a single file. It is a coupled state across modalities.
- ▶ AlphaFold provides the starting local structure.
- ▶ Cryo-EM fixes the global arrangement and lattice consistency.
- ▶ LAMMPS evolves the periodic unit-cell state.
- ▶ SAXS provides observable-space evaluation and back-calculation targets.

Example Protein Set

SASDXF6

Measured: scattering.dat
Context: metadata, models
Role: aligned $I(q)$ sample

SASDYY5

Measured: scattering.dat
Optional: pr.dat
Role: curve comparison +
real-space check

SASDC44

Measured: scattering.dat
Context: metadata, model
links
Role: shared
evaluation-space entry

Every downloaded protein enters the same semantic pipeline: measured curve first, optional attachments second, evaluation layer unchanged.

IgG Modalities

AlphaFold

Input: sequence, confidence, PAE

Output: initial monomer geometry

Role: structural prior

Cryo-EM + LAMMPS

Input: map, fit restraints, unit cell

Output: constrained MD state

Role: dynamic physical refinement

SAXS

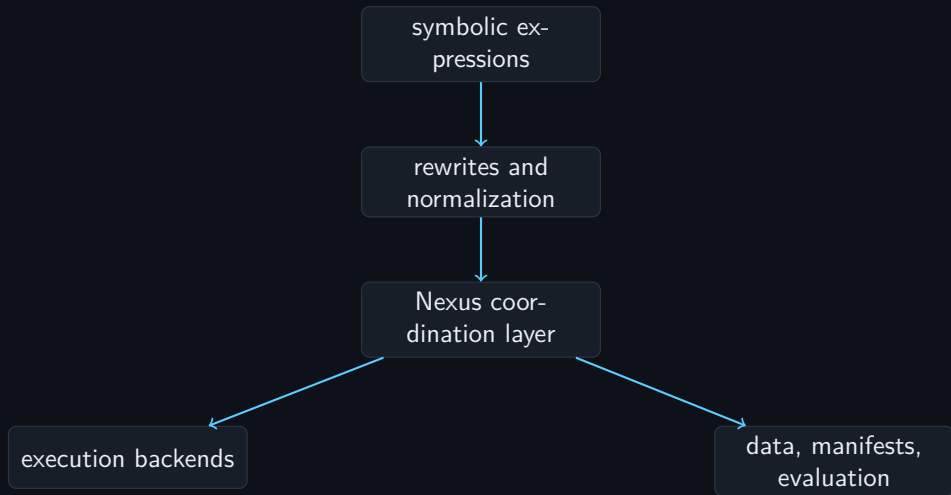
Input: measured or back-calculated curves

Output: $I(q)$, optional $P(r)$, PCA features

Role: observable-space evaluation

IgG is the template for sample structuring: multiple modalities, one object identity, one evaluation layer.

Architecture



Nexus Concept

Definition

Nexus is the operational membrane of Calyr: it connects symbolic intent, engine execution, and data evaluation without changing the meaning level of the scientific object.

- ▶ It does not introduce a second language for runtime control.
- ▶ It binds one object graph to environments, engines, inputs, and outputs.
- ▶ It records each transition as a manifest, so execution remains inspectable and reproducible.

Nexus as Declarative Control Plane

Nexus is not a logbook. Nexus is the control plane of scientific execution.

The system follows a strict declarative paradigm:

$$\text{State}_t \xrightarrow{\text{Nexus Declaration}} \text{Execution} \xrightarrow{\text{Results}} \text{State}_{t+1} \quad (1)$$

Every scientific action is defined as a transition between two fully specified Nexus states.

Correct Execution Paradigm

Incorrect (Imperative)

```
# manual workflow (forbidden)
edit lammps_input.script
run lammps
log results later
```

Correct (Declarative Nexus Flow)

```
# 1. modify Nexus declaration
git commit -m "pipeline:lammps:temp=310"

# 2. execute via Nexus
python -m nexus_engine lammps_experiment

# 3. results are automatically tracked
```

Rule: Nothing exists unless it is declared in Nexus.

Hierarchical Control Structure

$$\text{Nexus}_{root} \rightarrow \text{Nexus}_{domain} \rightarrow \text{Nexus}_{tool} \rightarrow \text{Execution} \quad (2)$$

This defines a multi-level control system:

- ▶ **Level 0: Root Nexus** — global policies, FAIR enforcement, scoring, publication gating
- ▶ **Level 1: Domain Nexus** — scientific semantics and normalization
- ▶ **Level 2: Tool Nexus** — execution bindings and environment definitions
- ▶ **Level 3: Experiment Layer** — declared scientific studies and comparisons

State Machine Interpretation

Nexus defines a state machine over scientific reality.

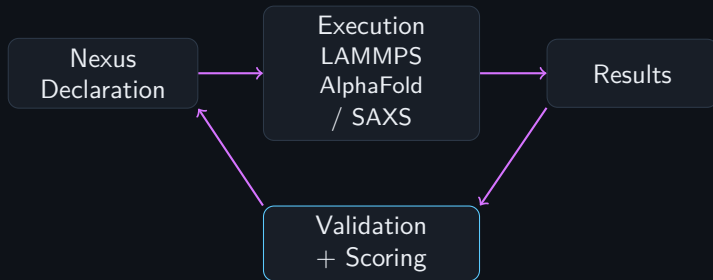
Each Git commit represents a discrete state:

$$S_t = \{\text{parameters, models, data, assumptions}\} \quad (3)$$

$$S_t \rightarrow S_{t+1} \quad (4)$$

The transition function is implemented by Nexus-controlled execution.

Closed-Loop Scientific System



Every result feeds back through validation before Nexus advances to the next declared state.

Separation of Concerns

Nexus = Declarative Layer (5)

Tools = Execution Layer (6)

This separation ensures:

- ▶ No hidden parameters
- ▶ Full reproducibility
- ▶ FAIR compliance by construction
- ▶ Cross-domain comparability

Formal Rule Set and Workflow

Formal Rule Set

1. No execution outside Nexus
2. No result without FAIR metadata
3. No experiment without declaration
4. No publication without validation

Operational Workflow

```
# Modify declaration
git commit -m "pipeline:lammeps:update-parameters"

# Execute
python -m nexus_engine experiment_name

# Store + track results
git commit -m "data:experiment_name:results"
```

Final Statement

**You do not run simulations.
You materialize Nexus states.**

PCA Intro for Both Regimes

Input to PCA

A collection of samples is mapped to a common numerical representation, for example downloaded SAXS curves on a fixed q -grid or aligned feature vectors from IgG simulation and analysis.

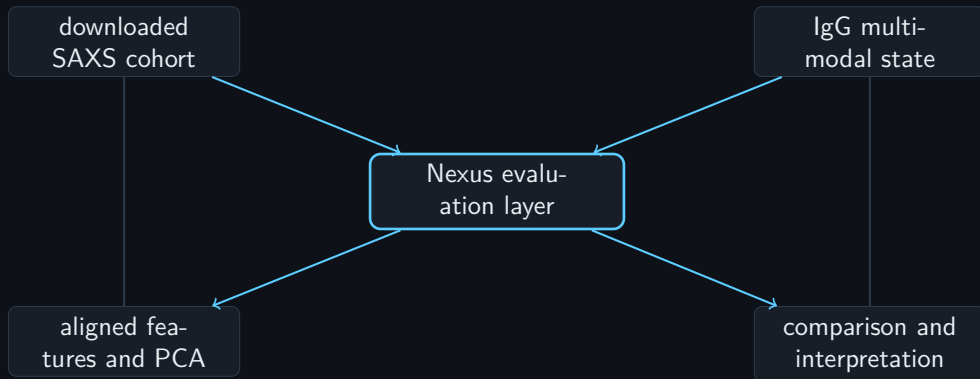
- ▶ one row per protein entry, state, frame, or condition
- ▶ one aligned feature vector per row
- ▶ PCA reveals dominant structural or observable variation modes

Interpretation

The principal components are not new raw data. They are a compact coordinate system for comparing measured cohort samples and structured IgG states without breaking sample identity.

In the repo workflow, PCA is only meaningful once observables or fitted parameter sets are dimensionally aligned.

Representation Flow



Nexus holds both broad downloaded cohorts and deeply structured multimodal samples, so new setups can enter the same evaluation plane without redesigning the language each time.

One Evaluation Level

Without Nexus

Model, script, raw output, and analysis drift into separate layers with different naming and different state.

- ▶ hard to compare runs
- ▶ analysis detached from provenance
- ▶ semantics collapse into files

With Nexus

The same semantic object is carried from setup to result, so data evaluation stays on one coordinated plane.

- ▶ one identity for run and result
- ▶ one manifest chain for provenance
- ▶ one level for simulation and analysis

Why We Do Not Need Dmax

Key point

Dmax belongs to the optional real-space $P(r)$ description. It is not required to place downloaded SAXS curves or IgG SAXS observables into a Nexus representation or to run PCA across aligned samples.

- ▶ PCA can operate directly on standardized $I(q)$ vectors.
- ▶ PCA can also operate on fitted parameter vectors when they have common length.
- ▶ Dmax is useful for $P(r)$ interpretation and sanity checks, but it is not a gatekeeper for representation learning.
- ▶ This keeps the evaluation layer closer to the measured data and avoids introducing an unnecessary extra assumption early in the pipeline.

Why It Fits Calyr

- ▶ **core/** supplies the model vocabulary.
- ▶ **engines/** provides realizations such as LAMMPS or SAXS execution.
- ▶ **experiments/** defines concrete scientific programs such as IgG.
- ▶ **data/** and **results/** become typed endpoints rather than loose folders.
- ▶ **nexus/** keeps model state, execution state, and evaluation state aligned instead of splitting them across ad hoc tools.
- ▶ The same framework can hold downloaded SAXS proteins, their embeddings, their fitted parameters, and their downstream comparisons on one semantic level.

Minimal Object Model

Symbolic form

$\text{Expr}(\text{Head}, \text{Args}, \text{Meta})$

- ▶ Head defines semantic type.
- ▶ Args carries ordered structure.
- ▶ Meta holds provenance, bindings, constraints, and evaluation context.

Outcome

One language layer can describe equations, workflows, engine calls, and analysis contracts without changing representation.

Near-Term Build

1. Define the symbolic core and canonical node types.
2. Add rewrite rules and validation passes.
3. Bind syntax objects to Nexus pipelines and managed environments.
4. Evaluate data products through the same object graph used for execution.
5. Emit reproducible manifests for execution and analysis.

goal: a compact language substrate for reproducible computational science inside Calyr

Calyr_ai_syntax: Concept and Architecture

symbolic clarity before execution complexity

CALYRAI × BOKU VIENNA

Prepared for selective institutional presentation use

Calyrai Project

Three Core Computational Engines

AlphaFold

Input: sequence, PAE, confidence

Output: initial monomer geometry

Role: structural prior for IgG

LAMMPS

Input: unit cell, restraints, force field

Output: constrained MD trajectory

Role: dynamic physical refinement

OpenMM

Input: topology, initial coords

Output: GPU-accelerated trajectory

Role: flexible force field MD on cluster

All three engines are Nexus-registered tools. No engine runs outside the Nexus control plane.

Engines as Nexus-Bound Tools

A computational engine that is not declared in Nexus does not exist in Calyrai.

- ▶ Each engine is registered in `nexus/core/registry.yaml`
- ▶ Invocation is always through a declared pipeline step
- ▶ The engine changes; the declaration language does not

Registry

```
valid_tools: ["alphafold", "lammps", "openmm", "asc_submit"]
```

ASC: Remote Execution Surface

Cluster Parameters

Host: <asc-host>

Scheduler: SLURM

Max runtime: 72 hours

User: <cluster-user>

Project: <cluster-project>

Home: /home/<project>/<user>

Data: /gpfs/data/<project>/<user>

Declaration unchanged

The same Nexus pipeline that runs locally runs on ASC. Only the executor field changes.

Execution surface = *where*.

Workflow declaration = engine-agnostic.

Installation Pipeline on ASC

Pattern: check → install → verify

```
# Full installation pipeline
nexus run asc_install_computational_engines

# Stage-scoped
nexus run asc_install_computational_engines \
  --stage check_availability
nexus run asc_install_computational_engines \
  --stage verify_installation
```

Presentation shorthand uses the ASC naming for the remote execution surface and installation

FAIR for Remote Runs

- ▶ **Findable:** every ASC job has a `run_id` derived from pipeline name + timestamp
- ▶ **Accessible:** output paths declared in `nexus/core/asc_config.yaml` and stable
- ▶ **Interoperable:** environment spec is a versioned conda YAML in Git
- ▶ **Reusable:** same declaration reruns identically on any compatible cluster

The scientific workflow remains constant. The machine changes.

Nexus Data Layer

Declarative query

```
{entity:  scattering_curves,  
  filter: {quality:  high}}
```

↓ *translated to:*

Resolved SQL

```
SELECT * FROM scattering_curves  
WHERE quality_score >= 0.8;
```

FAIR query log

```
query_hash: a55bef0...  
timestamp: 2026-03-26T...  
reproducible: true
```

Same query $\Rightarrow$ same hash.

Data access is as auditable as code.

From Simulation to Construction

Forward (simulation)

$$y = F(x)$$

Given a state x , predict outcome y .

Inverse (construction)

given y^* : find x such that $F(x) \approx y^*$

Choose interventions to produce a target state.

Construction is control, not only prediction.

The Qualitative Jump

Level	Core question
Description	What is?
Simulation	What happens if...?
Construction	What must I do so that...?

- ▶ Many classical fitting systems are simulatable but not constructable.
- ▶ Degenerate fits and non-identifiability block stable inversion.
- ▶ Calyrai addresses this with constraints + closed-loop execution.

Nexus as Construction Machine

Target → constrained inversion → candidates → experiment → feedback → update

- ▶ **Representations:** real space, Fourier space, energy landscape
- ▶ **Constraints:** stability, geometry, interfaces, packing
- ▶ **Loop:** propose, test, learn, iterate

Inference → Control → Design is the operational transition.

Limits and Honest Boundaries

- ▶ Not every system is fully constructable.
- ▶ Limits: high dimensionality, incomplete data, nonlinear couplings, non-identifiability.
- ▶ Therefore: iterative approximation, probabilistic candidates, robust correction layers.

Simulatability enables prediction. Constructability enables control. Closed loops make both operational.

Double-Saturated Langmuir Model (SPR)

Two coupled adsorption populations

$$R(t, C) = R_0(C) + Q_1\theta_1(t, C) + Q_2\theta_2(t, C)$$

Dynamics

$$\left(\frac{d}{dt} + \lambda_1\right)^{k_1} \theta_1 = \lambda_1^{k_1-1} \alpha_1 C^{m_1} (1 - \theta_1)$$

$$\left(\frac{d}{dt} + \lambda_2\right)^{k_2} \theta_2 = \lambda_2^{k_2-1} \alpha_2 (C\theta_1^n)^{m_2} (1 - \theta_2)$$

► θ_1 : primary adsorption population

► λ_i : characteristic step timescales

Equilibrium Isotherm and Experimental Validation

Steady-state limit

$$\theta_{1,\infty}(C) = \frac{K_1 C^{m_1}}{1 + K_1 C^{m_1}}, \quad \theta_{2,\infty}(C) = \frac{(K_2 C \theta_{1,\infty}^n)^{m_2}}{1 + (K_2 C \theta_{1,\infty}^n)^{m_2}}$$

Resulting isotherm

$$R_\infty(C) = R_0(C) + Q_1 \theta_{1,\infty}(C) + Q_2 \theta_{2,\infty}(C)$$

Equilibrium Isotherm and Experimental Validation

- ▶ First plateau: primary adsorption
- ▶ Second plateau: gated secondary binding
- ▶ Explains the shoulder and double saturation observed in SPR

Interpretation

$$K_1 = \frac{\alpha_1}{\lambda_1}, \quad K_2 = \frac{\alpha_2}{\lambda_2}$$

Large K_i means stronger effective occupation at equilibrium.

Experimental Validation and Reference

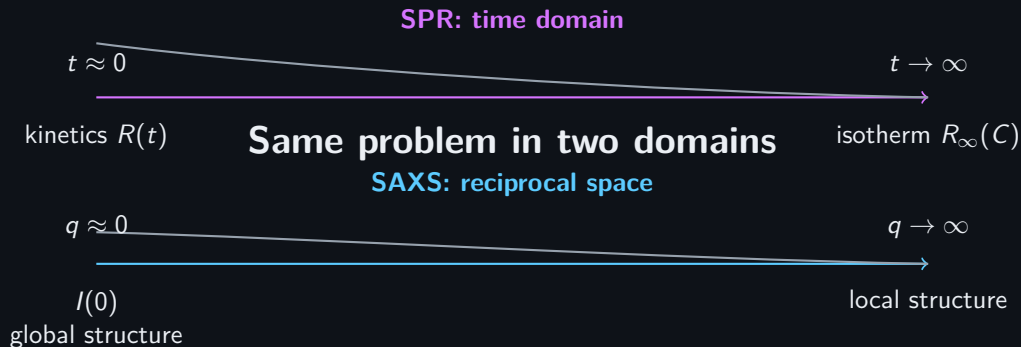
Experimental reference

R. Tscheliessnig et al., *Journal of Chromatography A* (2024)

<https://www.sciencedirect.com/science/article/pii/S002196732400476X?via%3Dihub>

SAXS + adsorption isotherms were combined to resolve antibody–ligand structure and binding modes.

One Model Must Span the Full Domain



Key idea

Neither SPR nor SAXS can be interpreted locally. Both require a model that spans the entire domain.

From Model Fitting to PCA-Based Prediction

Abstract observables

SPR is observed as a weighted response superposition:

$$R(t, C) = R_0(C) + \sum_{i=1}^2 Q_i \theta_i(t, C)$$

SAXS is observed through the mixed pair-distance distribution and its Fourier projection:

$$PDD_{\text{frac}}(r) \approx \sum_{j=1}^M f_j P_j(r), \quad I(Q) = \text{FFT}(PDD_{\text{frac}}(r)) [Q]$$

Both modalities require one model spanning the full domain, not local point-wise interpretation.

From Model Fitting to PCA-Based Prediction

SAXS observable from state fractions

For structural states with fractions f_j and $\sum_j f_j = 1$,

$$PDD_{\text{frac}}(r) \approx \sum_{j=1}^M f_j P_j(r), \quad I_{\text{mix}}(Q) = \text{FFT}(PDD_{\text{frac}}(r)) [Q]$$

$$I_j(Q) = \text{FFT}(P_j(r)) [Q]$$

SAXS gives the Fourier-space signature of the weighted state mixture.

From Model Fitting to PCA-Based Prediction

Proposed extension

Use PCA across aligned features from both modalities:

$$\{I_{\text{mix}}(Q)\}_{\text{SAXS}}, \quad \{R(t)\}_{\text{SPR}}, \quad \{\theta\text{-parameters}, f_j\}$$

Goal: fast prediction instead of repeated fitting.

From Model Fitting to PCA-Based Prediction

What PCA should learn

The representation should encode both measured dynamics and structure-derived mixtures on one aligned domain.

- ▶ Build a joint representation of protein state, kinetics, and mixture composition
- ▶ Map experimental systems into one PCA space instead of fitting each case in isolation
- ▶ Predict adsorption behavior, structural class, and multi-binding regimes

Key idea

SAXS provides Fourier-space structural constraints from the weighted fractions.

SPR provides kinetics and equilibrium occupations.

PCA provides the predictive layer.

Wrap-Up and Open Items

Current status

The concept deck now links symbolic language, Nexus execution, SPR kinetics, SAXS observables, and PCA-based prediction in one narrative.

- ▶ Code and homepage presentation tested locally
- ▶ ASC transfer and operational rollout in the coming days
- ▶ Targeted improvements to the presentation and execution workflow remain open

Next step: move from concept validation to operational transfer and refinement.